



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO.



- **Alumno:**

Rodriguez Rosales Diego Guillermo.

- **Num. Control:**

#222310306.

- **Docente:**

Jesus Salas Marin.

- **Materia:**

Desarrollo de Aplicaciones para Dispositivos Móviles.

- **Clave:**

4Y8A.

- **Actividad:**

Lección La 1er aplicación de Flutter.

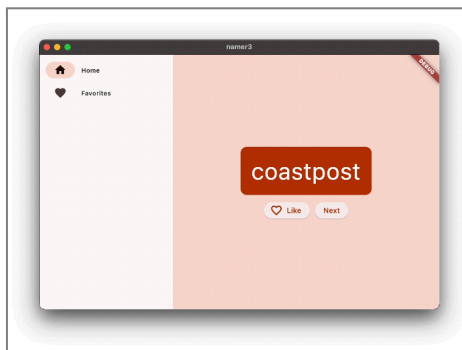
11-Mayo-2026.





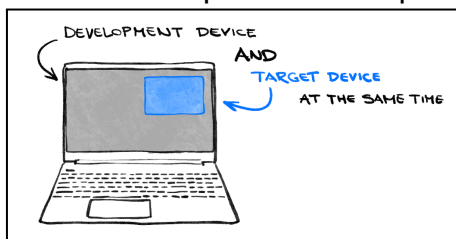
Capítulo 1. Introducción.

1. En esta práctica se realizará la siguiente aplicación con Flutter ya que es el kit de herramientas de IU de Google para crear aplicaciones que funcionen en dispositivos móviles, la web y computadoras de casa.



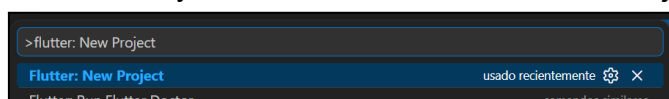
Capítulo 2. Configuración en el entorno de Flutter.

1. En primer lugar, Flutter debe estar instalado en el equipo.
2. Usaremos VS Code en Windows para hacer la práctica.

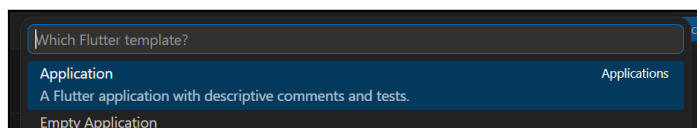


Capítulo 3. Creación del proyecto.

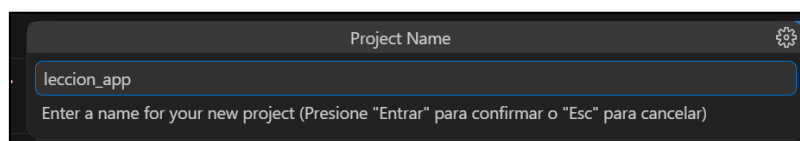
1. Iniciamos usando el comando Ctrl+Shift+P para abrir el buscador y escribimos "flutter new" y seleccionamos "Flutter: New Project".



2. Y seleccionamos la opción de Application y elegimos la carpeta en donde se creará el proyecto (es donde deseemos donde se guardará la carpeta).

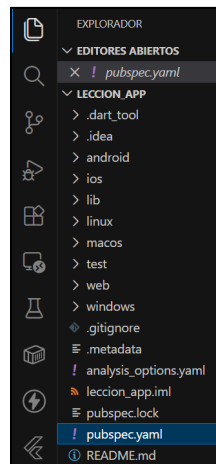


3. Para seguir, asignamos el nombre del proyecto, en este caso será "leccion_app".

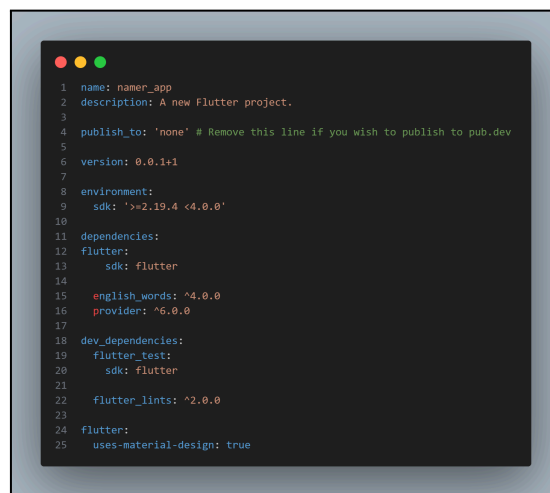




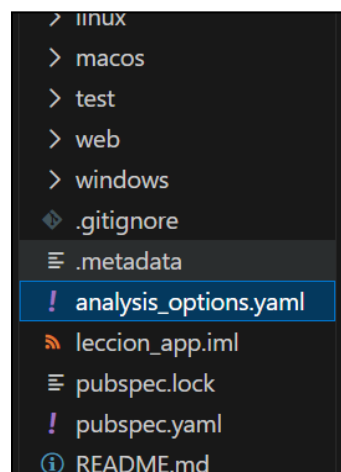
4. Así se verá la estructura de la carpeta eh iremos al archivo “pubspec.yaml”.



- Y en el archivo reemplazamos el contenido con el siguiente código.



5. Ahora iremos al archivo “analysis_options.yaml”.

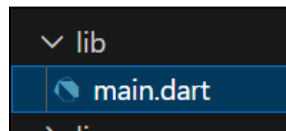




- Y modificamos el archivo con el siguiente código.

```
1 include: package:flutter_lints/flutter.yaml
2
3
4 linter:
5   rules:
6     prefer_const_constructors: false
7     prefer_final_fields: false
8     use_key_in_widget_constructors: false
9     prefer_const_literals_to_create_immutables: false
10    prefer_const_constructors_in_immutables: false
11    avoid_print: false
```

6. Iremos al archivo más importante, en este caso es “main.dart”.



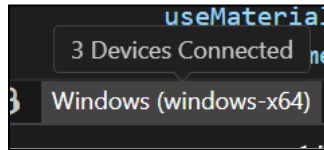
- En el archivo, ponemos la siguiente estructura para nuestra aplicación.

```
1 import 'package:english_words/english_words.dart';
2 import 'package:flutter/material.dart';
3 import 'package:provider/provider.dart';
4
5
6 void main() {
7   runApp(MyApp());
8 }
9
10
11 class MyApp extends StatelessWidget {
12   const MyApp({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return ChangeNotifierProvider(
17       create: (context) => MyAppState(),
18       child: MaterialApp(
19         title: 'Namer App',
20         theme: ThemeData(
21           useMaterial3: true,
22           colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
23         ),
24         home: MyHomePage(),
25       ),
26     );
27   }
28 }
29
30
31 class MyAppState extends ChangeNotifier {
32   var current = WordPair.random();
33 }
34
35
36 class MyHomePage extends StatelessWidget {
37   @override
38   Widget build(BuildContext context) {
39     var appState = context.watch<MyAppState>();
40
41     return Scaffold(
42       body: Column(
43         children: [
44           Text('A random idea:'),
45           Text(appState.current.asLowerCase),
46         ],
47       ),
48     );
49   }
50 }
51
52 }
```

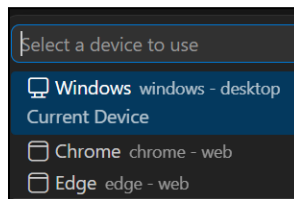


Capítulo 4. Agregar un botón.

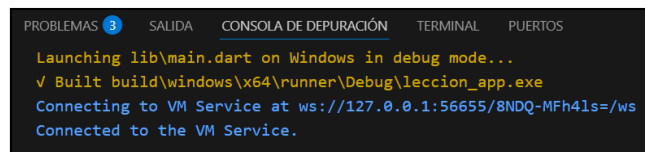
1. Para iniciar nos vamos a la barra de abajo y cambiamos el dispositivo de destino al de nuestro equipo.



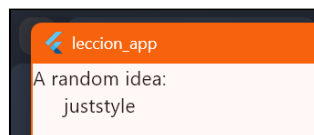
- Seleccionamos en este caso el escritorio de Windows.



2. Ya seleccionado el escritorio, abrimos la terminal con la opción  en la parte superior de VS Code e iniciamos la depuración.
 - *Nota: Si queremos iniciarlo por comando podemos usar “flutter run lib/main.dart”.*



- Así se verá la aplicación ya iniciada/depurada.



3. En la parte inferior del código “main.dart” agregamos la siguiente cadena del primer objeto Text y guardamos el archivo con Ctrl+S.





- De forma inmediata la aplicación refleja los cambios.

```
A random AWESOME idea:  
juststyle
```

4. A continuación, agregamos un botón en la parte superior del archivo, agregamos la clase “MyApp” con la extensión “StatelessWidget”.

```
Run | Debug | Profile  
void main() {  
  runApp(MyApp());  
}
```

5. Así se verá la clase.

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return ChangeNotifierProvider(  
      create: (context) => MyAppState(),  
      child: MaterialApp(  
        title: 'Namer App',  
        theme: ThemeData(  
          useMaterial3: true,  
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),  
        ), // ThemeData  
        home: MyHomePage(),  
      ), // MaterialApp  
    ); // ChangeNotifierProvider  
  }  
}
```

6. Con la clase “MyAppState” definimos el estado de la app, usando la extensión “ChangeNotifier”, lo que hace que pueda notificar a otros acerca de sus propios cambios

```
class MyAppState extends ChangeNotifier  
  var current = WordPair.random();
```



7. Para terminar tenemos el elemento “MyHomePage”, el cual se encarga de actualizar la interfaz mediante el método build(), que reconstruye los widgets cada vez que hay cambios en la aplicación. Además, administra el estado actual usando watch, y dentro de su estructura utiliza widgets como Scaffold para la base de la app y Column para organizar los elementos verticalmente. También incluye widgets de texto que muestran información del estado actual, como las palabras generadas con WordPair, demostrando cómo Flutter organiza y actualiza la interfaz de manera dinámica y ordenada.

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) { // -1  
    var appState = context.watch<MyAppState>(); // -2  
  
    return Scaffold( // -3  
      body: Column( // -4  
        children: [  
          Text('A random AWESOME idea:'), // -5  
          Text(appState.current.asLowerCase), // -6  
          ElevatedButton(  
            onPressed: () {  
              print('Botón presionado');  
            },  
            child: Text('Presionar'),  
          ) // ElevatedButton  
        ], // -7  
      ), // Column  
    ); // Scaffold  
  }  
}
```

8. Volvemos a “MyAppState” y agregamos el método “getNext()”.

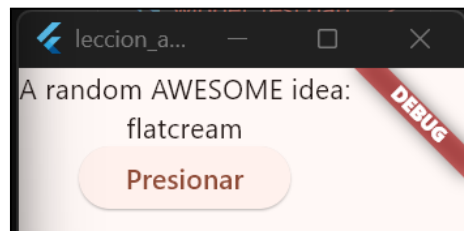
```
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  
  void getNext(){  
    current = WordPair.random();  
    notifyListeners();  
  }  
}
```

9. El método “getNext()” asignará el elemento current con un nuevo WordPair aleatorio. También llamará a notifyListeners() (un método de ChangeNotifier) que garantiza que se notifique a todo elemento que esté mirando a MyAppState. Lo que resta es llamar al método getNext desde la devolución de llamada del botón.

```
ElevatedButton(  
  onPressed: () {  
    appState.getNext();  
  },  
  child: Text('Presionar'),  
) // ElevatedButton
```



- Guardamos y ejecutamos la app, y ahora deberá generar nuevas palabras cada vez que presionamos el botón.

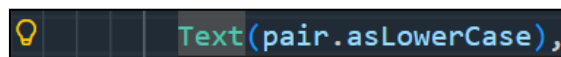


Capítulo 5. La app más atractiva.

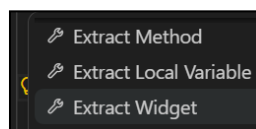
1. Volveremos al widget de “MyHomePage” y cambiamos el código de la siguiente manera.

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    var appState = context.watch<MyAppState>();  
    var pair = appState.current;  
  
    return Scaffold(  
      body: Column(  
        children: [  
          Text('A random AWESOME idea:'),  
          Text(pair.asLowerCase),  
  
          ElevatedButton(  
            onPressed: () {  
              appState.getNext();  
            },  
            child: Text('Presionar'),  
          )  
        ],  
      ),  
    );  
  }  
}
```

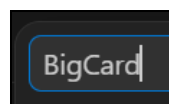
2. Lo que sigue es ir refactorizador Text y dar click en el foco de la izquierda.



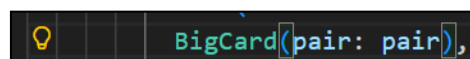
- Elegimos la opción “Extract Widget”.



- En el buscador que se abrirá ponemos “BigCard”.



- Cambiará la etiqueta de “Text” a “BigCard”.





- Y se verá la variable de la siguiente manera.

```
class BigCard extends StatelessWidget {
  const BigCard({
    super.key,
    required this.pair,
  });

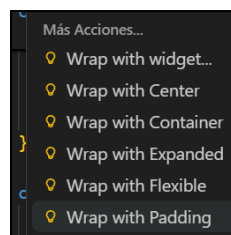
  final WordPair pair;

  @override
  Widget build(BuildContext context) {
    return Text(pair.asLowerCase);
  }
}
```

3. Ahora, agregaremos una tarjeta, en la clase “BigCard” buscamos el método “build()”, seleccionamos la variable “Text” y click al foco izquierdo.

```
return Text(pair.asLowerCase);
```

- Seleccionamos la opción “Wrap with Padding”.



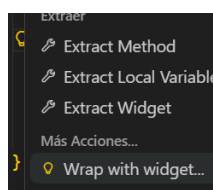
- Y nos dará el siguiente widget.

```
@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.all(20),
    child: Text(pair.asLowerCase),
  ); // Padding
}
```

4. A continuación, buscamos el widget Padding y damos click en el foco amarillo.

```
return Padding(
```

- Seleccionamos la opción “Wrap with widget”.

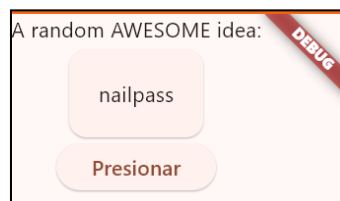




- Nos dará error al cambiar, pero simplemente cambiamos la variable “Padding” por “Card”.

```
@override
Widget build(BuildContext context) {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Text(pair.asLowerCase),
    ), // Padding
  ); // Card
```

- Guardamos y se verá de la siguiente manera la aplicación.



5. En esta sección, cambiamos las 2 líneas de código que se muestran en la imagen.

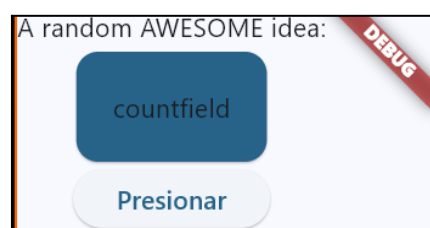
- Estas dos líneas realizan varias tareas:
 - Primero, el código solicita el tema actual de la app con Theme.of(context).
 - Luego, el código define el color de la tarjeta de modo que sea el mismo que el de la propiedad colorScheme del tema. El esquema de colores contiene varios de ellos, y primary es el más destacado y el que define el color de la app.

```
@override
Widget build(BuildContext context) {
  final theme = Theme.of(context); //esto

  return Card(
    color: theme.colorScheme.primary, //y esto
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Text(pair.asLowerCase),
    ), // Padding
  ); // Card
```

6. Ahora la tarjeta estará pintada de un color primario, pero simplemente en esta etiqueta elegimos el color a nuestro gusto.

```
colorScheme: ColorScheme.fromSeed(seedColor: const Color.fromARGB(177, 35, 167, 237)),
```





7. En esta sección cambiamos lo que se ve en la imagen:
- Este cambio utiliza “theme.textTheme” para acceder a los estilos de texto de la aplicación, como títulos, leyendas y texto estándar. En este caso, se usa “displayMedium”, un estilo pensado para textos cortos e importantes. Además, Dart cuenta con seguridad contra valores nulos, por lo que se utiliza el operador ! para indicar que “displayMedium” no será nulo y puede utilizarse con seguridad.

```
@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  //agregamos esto
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );

  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),
      //y agregamos esto
      child: Text(pair.asLowerCase, style: style),
    ), // Padding
  ); // Card
}
```

- La aplicación tendrá la siguiente estructura.



8. Ahora, los lectores de pantalla pronuncian correctamente cada par de palabras generado, pero la IU sigue siendo igual. Observa esto en acción usando un lector de pantalla en tu dispositivo.

```
@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );

  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),
      //y agregamos esto
      child: Text(
        pair.asLowerCase,
        style: style,
        semanticsLabel: "${pair.first} ${pair.second}",
      ), // Text
    ), // Padding
  ); // Card
}
```



9. Ahora que el par de palabras tiene un mejor estilo visual, el siguiente paso es centrarse en la pantalla. Para lograrlo, se modifica el widget Column dentro del método build() de MyHomePage, cambiando la alineación predeterminada de sus elementos, que normalmente se colocan en la parte superior.

```
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

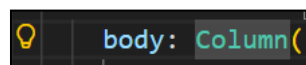
    return Scaffold(
      body: Column(
        mainAxisAlignment: MainAxisAlignment.center, //est
        children: [
          Text('A random AWESOME idea:'),
          BigCard(pair: pair),

          ElevatedButton(
            onPressed: () {
              appState.getNext();
            },
            child: Text('Presionar'),
          ) // ElevatedButton
        ],
      ), // Column
    ); // Scaffold
  }
}
```

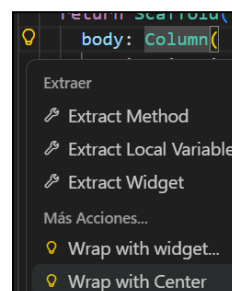
- Se verá de la siguiente manera la aplicación.



10. En esa misma sección vamos a la variable “body” y cambiamos el método “Column”.



- Y seleccionamos la opción “Wrap with Center”.





- Se verá el código de la siguiente manera.

```
return Scaffold(  
  body: Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  
        Text('A random AWESOME idea:'),  
        BigCard(pair: pair),  
  
        ElevatedButton(  
          onPressed: () {  
            appState.getNext();  
          },  
  
          child: Text('Presionar'),  
        ) // ElevatedButton  
      ],  
    ),  
  ),  
);
```

- Y de este modo es como se verá la aplicación.

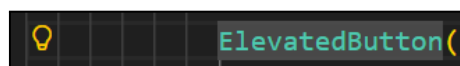


Capítulo 6. Agregar funcionalidad.

1. Volvemos a la clase “MyAppState” y agregamos lo que se muestra en la imagen.

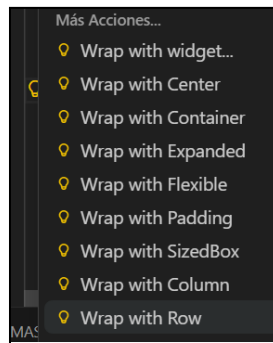
```
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  
  void getNext(){  
    current = WordPair.random();  
    notifyListeners();  
  }  
  //Agregamos esto  
  var favorites = <WordPair>[];  
  
  void toggleFavorite() {  
    if (favorites.contains(current)){  
      favorites.remove(current);  
    } else {  
      favorites.add(current);  
    }  
    notifyListeners();  
  }  
}
```

2. Vamos a la variable “ElevatedButton” y clickeamos el foco.





- Y seleccionamos la opción “Wrap with Row”.



- Y se verá de la siguiente manera la estructura.

```
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    return Scaffold(
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('A random AWESOME idea:'),
            BigCard(pair: pair),
            SizedBox(height: 10,),

            Row(
              children: [
                ElevatedButton(
                  onPressed: () {
                    appState.getNext();
                  },
                  child: Text('Presionar'),
                ), // ElevatedButton
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

3. En este paso se agrega un segundo botón a MyHomePage usando “ElevatedButton.icon()”, lo que permite mostrar un botón con ícono. El ícono cambia dependiendo de si el par de palabras actual ya está marcado como favorito. También se utiliza “SizedBox” para dejar espacio y separar visualmente ambos botones.

- IconData:

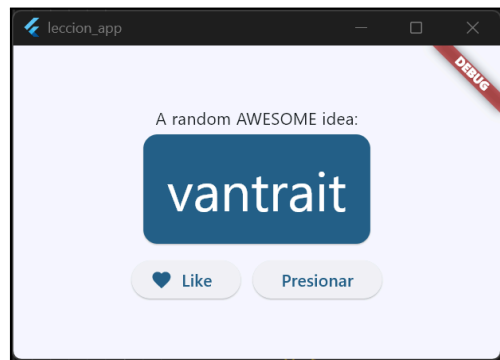
```
IconData icon;
if (appState.favorites.contains(pair)){
  icon = Icons.favorite;
} else {
  icon = Icons.favorite_border;
}
```



- ElevatedButton:

```
ElevatedButton.icon(  
  onPressed: (){  
    appState.toggleFavorite();  
  },  
  icon: Icon(icon),  
  label: Text('Like'),  
), // ElevatedButton.icon  
  SizedBox(width: 10),
```

- Así se verá la aplicación en este momento.



Capítulo 7. Agregar un riel de navegación.

1. Selecciona todo lo que está en MyHomePage (La clase completa), bórralo y reemplázalo con el siguiente código:

```
import 'package:english_words/english_words.dart';  
  
import 'package:flutter/material.dart';  
  
import 'package:provider/provider.dart';  
  
void main() {  
  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
  
    return ChangeNotifierProvider(  
  
      create: (context) => MyAppState(),
```



```
child: MaterialApp(  
  
  debugShowCheckedModeBanner: false,  
  
  title: 'Namer App',  
  
  theme: ThemeData(  
  
    useMaterial3: true,  
  
    colorScheme: ColorScheme.fromSeed(  
  
      seedColor: const Color.fromARGB(177, 35, 167, 237),  
  
    ),  
  
  ),  
  
  home: const MyHomePage(),  
  
),  
  
);  
  
}
```

```
class MyAppState extends ChangeNotifier {  
  
  var current = WordPair.random();  
  
  
  var favorites = <WordPair>[];  
  
  
  void getNext() {  
  
    current = WordPair.random();  
  
    notifyListeners();  
  
  }  
  
  
  void toggleFavorite() {  
  
    if (favorites.contains(current)) {  
  
      favorites.remove(current);  
  
    } else {  
  
      favorites.add(current);  
  
    }  
  
  }  
  
}
```




```
        notifyListeners();
    }
}

class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: const [
                NavigationRailDestination(
                  icon: Icon(Icons.home),
                  label: Text('Home'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.favorite),
                  label: Text('Favorites'),
                ),
              ],
              selectedIndex: 0,
              onDestinationSelected: (value) {
                debugPrint('selected: $value');
              },
            ),
          ),
        ],
      ),
    ),
  ),
);
```



```
Expanded(  
  
  child: Container(  
  
    color: Theme.of(context).colorScheme.primaryContainer,  
  
    child: const GeneratorPage(),  
  
  ),  
  
),  
  
],  
  
),  
  
);  
}  
}
```

```
class GeneratorPage extends StatelessWidget {  
  
  const GeneratorPage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
  
    var appState = context.watch<MyAppState>();  
  
    var pair = appState.current;  
  
    IconData icon;  
  
    if (appState.favorites.contains(pair)) {  
  
      icon = Icons.favorite;  
  
    } else {  
  
      icon = Icons.favorite_border;  
  
    }  
  
    return Center(  
  
      child: Column(  
  
        mainAxisAlignment: MainAxisAlignment.center,  
  
        children: [  

```





```
const Text(  
  'A random AWESOME idea:',  
  style: TextStyle(fontSize: 20),  
)  
  
const SizedBox(height: 20),  
  
BigCard(pair: pair),  
  
const SizedBox(height: 20),  
  
Row(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: [  
    ElevatedButton.icon(  
      onPressed: () {  
        appState.toggleFavorite();  
      },  
      icon: Icon(icon),  
      label: const Text('Like'),  
    ),  
  
    const SizedBox(width: 10),  
  
    ElevatedButton(  
      onPressed: () {  
        appState.getNext();  
      },  
      child: const Text('Next'),  
    ),  
  ],  
)
```

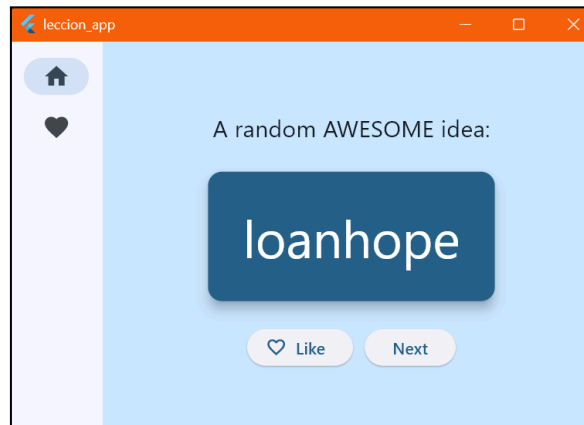


```
    ],  
  
    ),  
  
    );  
  }  
}  
)  
  
class BigCard extends StatelessWidget {  
  
  const BigCard({  
  
    super.key,  
  
    required this.pair,  
  
  });  
  
  final WordPair pair;  
  
  @override  
  Widget build(BuildContext context) {  
  
    final theme = Theme.of(context);  
  
    final style = theme.textTheme.displayMedium!.copyWith(  
  
      color: theme.colorScheme.onPrimary,  
  
    );  
  
    return Card(  
  
      color: theme.colorScheme.primary,  
  
      elevation: 8,  
  
      child: Padding(  
  
        padding: const EdgeInsets.all(30.0),  
  
        child: Text(  
  
          pair.asLowerCase,  
  
          style: style,  
  
          semanticsLabel: "${pair.first} ${pair.second}",  
  
        ),  
      ),  
    );  
  }  
}
```



```
),  
  
);  
  
}  
  
}
```

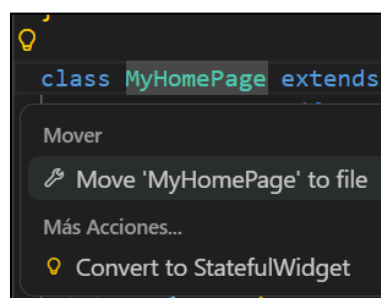
- Al guardar el progreso la UI visual ya estará lista.



2. Hasta este punto, el estado de la aplicación se maneja únicamente con “MyAppState”, por lo que los widgets estaban sin estado y dependían completamente de él para actualizarse. Sin embargo, ahora surge la necesidad de guardar y modificar valores propios, como selectedIndex del riel de navegación, ya que almacenar todo en “MyAppState” haría que el estado de la aplicación creciera demasiado y fuera más difícil de manejar.

```
// Agregamos esto  
var selectedIndex = 0;  
var selectedIndexInAnotherWidget = 0;  
var indexInYetAnotherWidget = 42;  
var optionASelected = false;  
var optionBSelected = false;  
var loadingFromNetwork = false;
```

3. Seleccionamos “MyHomePage”, clickeamos el foco y seleccionamos la opción “Convert to StatefulWidget”.





- Se verá de esta manera.

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key});  
  
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

4. El nuevo widget con estado tendrá la función de almacenar y controlar la variable `selectedIndex`, que indica el elemento seleccionado en la navegación. Para ello, se realizarán cambios en “_MyHomePageState” que permitirán actualizar y manejar este valor de forma dinámica.
- Estos cambios haremos:

```
class _MyHomePageState extends State<MyHomePage> {  
  
  var selectedIndex = 0; //Agregamos esto
```

```
  selectedIndex: selectedIndex, //Lo cambiamos  
  onDestinationSelected: (value) {  
    //Lo remplazamos  
    setState(() {  
      selectedIndex = value;  
    });  
  };
```

5. Coloca el siguiente código en la parte superior del método `build` de “_MyHomePageState”, justo antes de `return Scaffold`:

```
Widget page;  
  
switch (selectedIndex) {  
  case 0:  
    page = const GeneratorPage();  
    break;  
  
  case 1:  
    page = const Placeholder();  
    break;  
  
  default:  
    throw UnimplementedError('no widget for $selectedIndex');  
}
```



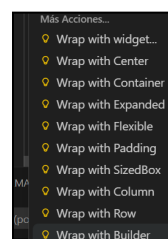
6. En esta parte del código se crea una variable llamada page de tipo Widget, la cual cambia según el valor de selectedIndex mediante una sentencia switch. Además, como “FavoritesPage” aún no está creada, se utiliza el widget Placeholder para mostrar un espacio temporal que indica que esa sección de la interfaz todavía no está terminada.

```
1 class _MyHomePageState extends State<MyHomePage> {  
2  
3   var selectedIndex = 0; //Agregamos esto  
4  
5   @override  
6   Widget build(BuildContext context) {  
7  
8     Widget page;  
9  
10    switch (selectedIndex) {  
11      case 0:  
12        page = const GeneratorPage();  
13        break;  
14  
15      case 1:  
16        page = const Placeholder();  
17        break;  
18  
19      default:  
20        throw UnimplementedError('no widget for $selectedIndex');  
21    }  
22  
23    return Scaffold(  
24      body: Row(  
25        children: [  
26          SafeArea(  
27            child: NavigationRail(  
28              extended: false,  
29              destinations: const [  
30                NavigationRailDestination(  
31                  icon: Icon(Icons.home),  
32                  label: Text('Home'),  
33                ),  
34                NavigationRailDestination(  
35                  icon: Icon(Icons.favorite),  
36                  label: Text('Favorites'),  
37                ),  
38              ],  
39              selectedIndex: selectedIndex,  
40              onDestinationSelected: (value) {  
41                setState(() {  
42                  selectedIndex = value;  
43                });  
44              },  
45            ),  
46          ),  
47          Expanded(  
48            child: Container(  
49              color: Theme.of(context).colorScheme.primaryContainer,  
50              child: page,  
51            ),  
52          ),  
53        ],  
54      ),  
55    ),  
56  );  
57 }  
58 }
```

7. Ahora iremos a la variable “return” y seleccionamos “Scaffold”.

```
return Scaffold(  
  )
```

- Y seleccionamos la opción “Wrap with Builder”.





- Cambiamos la variable.

```
return LayoutBuilder(
```

- Y cambiamos lo que contiene en la variable “builder”.

```
builder: (context, constraints) {  
  return Scaffold(
```

8. Ahora el código puede decidir si mostrar la etiqueta según las “constraints” actuales de la pantalla. Para lograrlo, se realiza un pequeño cambio en el método build de “_MyHomePageState”, permitiendo adaptar la interfaz de manera más dinámica.

```
return LayoutBuilder(  
  builder: (context, constraints) {  
    return Scaffold(  
      body: Row(  
        children: [  
          SafeArea(  
            child: NavigationRail(  
              extended: constraints.maxWidth == 600, //Aqui mijito  
              destinations: const [  
                NavigationRailDestination(  
                  label: Text('Home'),  
                  icon: const Icon(Icons.home),  
                ),  
                NavigationRailDestination(  
                  label: Text('Messages'),  
                  icon: const Icon(Icons.messages),  
                ),  
                NavigationRailDestination(  
                  label: Text('Ahoj'),  
                  icon: const Icon(Icons.a_hoj),  
                ),  
                NavigationRailDestination(  
                  label: Text('こんにちは'),  
                  icon: const Icon(Icons.konnichiwa),  
                ),  
              ],  
            ),  
          ),  
        ],  
      ),  
    ),  
  ),  
)
```

- Cambios realizados en la aplicación:

<https://drive.google.com/file/d/1lIOzckBKfcnXOIMXvuyPC4U6whEDhqBd/view?usp=sharing>

Capítulo 8. Agregar una nueva página.

1. Lo que sigue es solo una manera de implementar la página de favoritos. La forma en que se implementará te inspirará a que juegues con el código: mejora la IU y personalízala.

```
), // Row  
const Text(  
  'Messages:',  
  style: TextStyle(fontSize: 18),  
) , // Text  
...['Hello', 'Ahoj', 'こんにちは']  
  .map((msg) => Text(msg))  
  .toList(),  
],
```




2. Esta es la clase “FavoritesPage” nueva:
 - Este widget obtiene el estado actual de la aplicación y verifica si existen elementos favoritos. Si no hay favoritos, muestra un mensaje indicando “No favorites yet”; en caso contrario, presenta una lista desplegable con un resumen de favoritos y un “ListTile” para cada elemento. Finalmente, solo queda reemplazar el widget Placeholder por “FavoritesPage” para completar la funcionalidad.

```
class FavoritesPage extends StatelessWidget {
  const FavoritesPage({super.key});

  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();

    if (appState.favorites.isEmpty) {
      return const Center(
        child: Text('No favorites yet.'),
      ); // Center
    }

    return ListView(
      children: [
        Padding(
          padding: const EdgeInsets.all(20),
          child: Text(
            'You have ${appState.favorites.length} favorites:',
          ), // Text
        ), // Padding

        for (var pair in appState.favorites)
          ListTile(
            leading: const Icon(Icons.favorite),
            title: Text(pair.asLowerCase),
          ), // ListTile
      ],
    ); // ListView
  }
}
```

- **Y con esto ya está todo listo, enlace de demostración:**
https://drive.google.com/file/d/1fPnXbQYtG5M6AqxtD4O_IGIBHLRGR823/view?usp=sharing



Conclusión.

Durante el desarrollo de esta aplicación en Flutter se aprendió a utilizar widgets básicos y avanzados para construir interfaces dinámicas e interactivas. También se implementó el manejo de estado mediante Provider, permitiendo actualizar la información en tiempo real dentro de la aplicación.

Además, se trabajó con componentes como NavigationRail, botones, tarjetas y listas, logrando una aplicación capaz de generar palabras aleatorias y guardar elementos favoritos. Este proyecto permitió comprender mejor la estructura de Flutter, la organización del código y la creación de interfaces modernas y responsivas.